

Demo 3 — Head-to-head with the lasso

Same data, same train/validation split, three numbers each

Table of contents

Simulation	1
Lasso	2
R	2
Python	2
COMBSS	2
R	2
Python	3
Summary	3
Khan SRBCT	3
Multinomial lasso	3
R	3
Python	4
COMBSS	4
R	4
Python	4
Summary	4
What about MIO?	5
Bottom line	5

R code is executed; outputs below are real. Click the **Python** tab for the equivalent script.

The previous two demos showed COMBSS in isolation. This page runs it side-by-side with the lasso on the same datasets so the trade-offs become tangible.

We compare on three numbers: how many features each method selects, how it does out-of-sample, and how long the fit takes.

Simulation

The same data-generating mechanism as [Demo 1](#) — $n = 300$, $p = 30$, five true predictors.

```

set.seed(102)
n <- 300; p <- 30
beta <- c(3, 2, 1.5, 1, 0.5, rep(0, p - 5))
x <- matrix(rnorm(n * p), n, p)
y <- as.numeric(x %*% beta + rnorm(n) * 0.5)
itr <- 1:200; iva <- 201:300

```

Lasso

R

```

library(glmnet)
cv_l <- cv.glmnet(x[itr, ], y[itr], alpha = 1)
beta_l <- as.numeric(coef(cv_l, s = "lambda.min"))[-1]
sel_l <- which(beta_l != 0)
pred_l <- predict(cv_l, newx = x[iva, ], s = "lambda.min")
mse_l <- mean((y[iva] - pred_l)^2)
c(selected = length(sel_l), val_mse = round(mse_l, 4))

```

```

selected  val_mse
14.0000   0.2586

```

Python

```

from sklearn.linear_model import LassoCV
import numpy as np

cv_l = LassoCV(cv=10).fit(x[itr], y[itr])
sel_l = np.where(cv_l.coef_ != 0)[0]
mse_l = np.mean((y[iva] - cv_l.predict(x[iva])) ** 2)
print({"selected": len(sel_l), "val_mse": round(mse_l, 4)})

```

COMBSS

R

```

library(combss)
fit <- combss(x[itr, ], y[itr],
              x_val = x[iva, ], y_val = y[iva],
              family = "gaussian", q = 10)
sel_c <- fit$subset_list[[fit$best_k]]
mse_c <- min(fit$mse_path)
c(selected = length(sel_c), val_mse = round(mse_c, 4))

```

```
selected  val_mse
5.0000    0.2222
```

Python

```
from combss import combss
fit = combss(x[itr], y[itr], X_val=x[iva], y_val=y[iva],
             family="gaussian", q=10)
sel_c = fit.subset_list[fit.best_k - 1]
mse_c = min(fit.mse_path)
print({"selected": len(sel_c), "val_mse": round(mse_c, 4)})
```

Summary

Method	Selected	True positives	Val MSE
Lasso (cv.glmnet, lambda.min)	14	5	0.2586
COMBSS	5	5	0.2222

COMBSS returns *exactly* the five true predictors; lasso typically overshoots in selected size while landing on a comparable validation MSE.

Khan SRBCT

Same train/test split as the [Khan SRBCT demo](#).

```
train <- read.csv("../data/Khan_train.csv")
test  <- read.csv("../data/Khan_test.csv")
x_train <- as.matrix(train[, -1]); y_train <- factor(train$y)
x_test  <- as.matrix(test[, -1]); y_test  <- factor(test$y)
```

Multinomial lasso

R

```
library(glmnet)
cv_l <- cv.glmnet(x_train, y_train, family = "multinomial", alpha = 1)
beta_l <- coef(cv_l, s = "lambda.min")
sel_l_khan <- sort(unique(unlist(lapply(beta_l, function(B) which(B[-1, 1] != 0)))))
pred_l <- predict(cv_l, newx = x_test, s = "lambda.min", type = "class")
acc_l_khan <- mean(pred_l == y_test)
c(selected = length(sel_l_khan), test_acc = round(acc_l_khan, 3))
```

```
selected test_acc
35         1
```

Python

```
from sklearn.linear_model import LogisticRegressionCV
import numpy as np

cv_l = LogisticRegressionCV(
    penalty="l1", solver="saga", multi_class="multinomial",
    Cs=20, cv=10, max_iter=5000
).fit(X_train, y_train)
sel_l = np.where(np.any(cv_l.coef_ != 0, axis=0))[0]
acc_l = (cv_l.predict(X_test) == y_test).mean()
print({"selected": len(sel_l), "test_acc": round(acc_l, 3)})
```

COMBSS

R

```
library(combss)
fit_khan <- combss(x_train, as.character(y_train),
                  x_val = x_test, y_val = as.character(y_test),
                  family = "multinomial", q = 20)
acc_c_khan <- max(fit_khan$accuracy_path)
c(selected = fit_khan$best_k, test_acc = round(acc_c_khan, 3))
```

```
selected test_acc
      12         1
```

Python

```
from combss import combss
fit_khan = combss(X_train, y_train.astype(str),
                  X_val=X_test, y_val=y_test.astype(str),
                  family="multinomial", q=20)
print({"selected": fit_khan.best_k,
      "test_acc": round(max(fit_khan.accuracy_path), 3)})
```

Summary

Method	Genes selected	Test accuracy
Multinomial lasso	35	1
COMBSS	12	1

COMBSS finds a far smaller, interpretable gene panel without sacrificing accuracy — the headline result of this dataset.

What about MIO?

Mixed-integer best subset (Bertsimas, King, Mazumder, 2016) is the natural exact comparator. It is computationally feasible on the simulation but does **not** scale to the Khan setting at $p = 2308$ in any reasonable wall-clock budget. For the simulation, an MIO run produces exactly the same five-feature selection as COMBSS; the difference is wall-clock time.

Bottom line

- **Simulation** — COMBSS recovers the support exactly; lasso overshoots; MIO matches COMBSS when it can run.
- **Khan** — COMBSS gives a small gene panel with perfect classification, far smaller than the lasso panel.
- **Runtime** — COMBSS is in the same league as a CV lasso; MIO is the slow option even where it works.